

claude-windows / edc346c2-92fc-4ad1-b556-c7ab955a5613

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\edc346c2-92fc-4ad1-b556-c7ab955a5613\subagents\workflows\wf_47347652-5ae\agent-a896d50a2685dff1a.jsonl`
- SHA-256: `ba575df98a607ed9a316dbc37d6ba9f10eb6933e523214c5d153ecd1e215e9de`
- Source modified: `2026-07-05T07:00:44+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_47347652-5ae`
- session_id: `edc346c2-92fc-4ad1-b556-c7ab955a5613`
```

Transcript

1. user (2026-07-05T06:58:05.871Z)

SHARED CONTEXT (verified live this session, 2026-07-05):

```
- Repos (local checkouts = current master):
  * ENGINE: C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer (GitHub Khelsutra/badminton-highlight-indexer)
  * SPA: C:\Users\avidu\Projects\khelsutra-guru\khelsutra (web/src is the Studio SPA; GitHub avidullu/khelsutra)
- The LIVE hosted control-plane is an OLDER build than local master: capabilities reports engine_version "0.1.0",
  and its capabilities JSON has NO max_upload_mb / max_part_mb fields (current master DOES expose them, added around khelsutra #153). So distinguish "current master code" (what you can read) from "deployed old-build behavior" and
  note where they likely diverge.
- Deployed control-plane config.json (base64-injected at boot) sets ONLY:
  {"deployment":{"profile":"cloud-serving","compute_target":"cloud_run"},"indexing":{"skip_ai_handoff":true},
  "storage":{"backend":"gcs","output_root":"gs://khelsutra-rally-serving","input_backend":"gcs",
  "input_root":"gs://khelsutra-rally-serving/videos"},"security":{"...edge auth...}}
  Note: it does NOT pin any default/segmenter, and does NOT override security.max_upload_mb or max_part_mb.
- Live capabilities from the deployed CP: default_segmenter="yolo_hybrid",
  wasb_weights_present=false,
  gpu.nvidia_smi_present=false, gemini_key_present=false, deployment.cloud_available=true,
  profile=cloud-serving.
- Architecture: the CP (Cloud Run service, 4Gi RAM, no GPU, /tmp is RAM-backed) serves the SPA +
  dispatches
  cloud jobs to an L4 GPU Cloud Run JOB "rally-worker" which HAS the WASB weights mounted
  (WASB_WEIGHTS=gcs/models/...).
- Config defaults known from code: SecurityConfig.max_upload_mb=4096 (MB whole-file),
  max_part_mb=95 (per part).
- The e2e/battery that validated the alpha ALWAYS passed segmenter:"wasb_hybrid" EXPLICITLY in
  /api/process.
```

A finder proposed this ROOT CAUSE for ISSUE 1 (the 2.68 GB upload ? HTTP 413):

```
{
  "root_cause": "On the hosted alpha the 2.68 GB (2749 MB) upload does NOT 413 at any ENGINE
  size check ? both engine-side guards pass: the whole-file cap is 4096 MB (2749 < 4096, so
  /upload/init's line-100 check passes) and the SPA slices the file into ~95 MB parts (max_part_mb
  default), which are under the per-part cap. The 413 is produced UPSTREAM of the app by the Cloud
  Run / Google-Front-End request-body limit (~32 MiB for a non-streamed HTTP/1 request), which
  each ~95 MB PUT part blows past. The Knative manifest (deploy/cloudrun/gen_service_yaml.py) sets
  no run.googleapis.com/use-http2 annotation, so ingress is HTTP/1 and every part carries a fixed
```

95 MB Content-Length ? rejected with 413 at the edge before FastAPI's upload_part handler ever runs. It fails on part 0, so no bytes reach the RAM-backed /tmp and the 507 free-space guard is never even reached.",

```
"evidence": [
  {
    "file": "backend/api/uploads.py",
    "line": 100,
    "detail": "upload_init whole-file 413: `if req.total_size_bytes is not None and
req.total_size_bytes > max_bytes` ? max_bytes = max_upload_mb*1MB = 4096 MB. 2749 MB < 4096 MB,
so this does NOT fire. The SPA DOES send total_size_bytes (client.ts:251), so the check is
exercised and passes."
  },
  {
    "file": "backend/api/uploads.py",
    "line": 58,
    "detail": "part_bytes = max_part_mb(95)*1MB. This is the size the client is TOLD to chunk
at; /upload/init returns it as max_part_mb (line 122). 95 MB parts."
  },
  {
    "file": "backend/api/uploads.py",
    "line": 146,
    "detail": "upload_part per-part 413: `if received > part_bytes` and line 151 running-total
`if st['bytes']+received > max_bytes`. A ~95 MB part equals part_bytes and never exceeds it;
running total tops out at 2749 MB < 4096 MB. Neither fires ? this handler is not the source, AND
on Cloud Run HTTP/1 the 95 MB body never reaches it."
  },
  {
    "file": "backend/api/uploads.py",
    "line": 109,
    "detail": "require_free_bytes(upload_dir, total_size_bytes=2749MB, stage='upload') runs at
/upload/init. On the 4Gi instance /tmp is RAM-backed tmpfs, so shutil.disk_usage reports memory-
bound free space; a 2.68 GB reservation would raise HTTP 507 (NOT 413). This is a latent SECOND
failure, but it is a 507 and ? critically ? only lands in the CURRENT-master build; it is not
the observed 413."
  },
  {
    "file": "backend/utils/freespace.py",
    "line": 77,
    "detail": "require_free_bytes raises status_code=507, never 413. So the free-space guard
cannot be the source of the observed 413."
  },
  {
    "file": "deploy/cloudrun/gen_service_yaml.py",
    "line": 85,
    "detail": "The service manifest sets minScale/maxScale/cpu-throttling/timeout but NO
`run.googleapis.com/use-http2` annotation ? Cloud Run ingress is HTTP/1, whose per-request body
limit (~32 MiB) is below the 95 MB part size. Each PUT part is 413'd by the Google Front End
before the app sees it."
  },
  {
    "file": "deploy/cloudrun/gen_service_yaml.py",
    "line": 152,
    "detail": "--memory default 4Gi for the control-plane service, and config writes uploads
under RALLY_APP_HOME=/tmp/apphome (line 82) i.e. /tmp/apphome/output/uploads on RAM-backed tmpfs
? so even if parts got through, a 2.68 GB assembled file cannot fit alongside the runtime in
4Gi."
  },
  {
    "file": "deploy/cloudrun/README.md",
    "line": 47,
    "detail": "Authors' own note: '/tmp is RAM-backed ? the pulled input video lives in
memory, so size --memory for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of
tmpfs).' The 4Gi service cannot hold a 2.68 GB upload in tmpfs; the JOB is 32Gi, the control-
plane SERVICE is only 4Gi."
  },
]
```

```

{
  "file": "web/src/api/client.ts",
  "line": 320,
  "detail": "SPA chunking: partMb = init.max_part_mb (95) else 95; partBytes = 95*1MB;
totalParts = ceil(2749/95) = 29 parts. Each part.slice is a fixed-size Blob ? fetch sets a 95 MB
Content-Length (line 328-329, body: chunk). 29 parts of 95 MB, first one already over the edge
limit."
},
{
  "file": "backend/main.py",
  "line": 379,
  "detail": "CURRENT-master /api/capabilities return dict does NOT include max_upload_mb or
max_part_mb (only api_version, engine_version, segmenters, ffmpeg, gpu, gemini_key_present,
wasb_weights_present, deployment, storage). So the client pre-flight guard is inert even on
master."
},
{
  "file": "web/src/components/IngestPanel.tsx",
  "line": 82,
  "detail": "SPA reads c.max_upload_mb from capabilities to set the instant client-side
reject. Because the engine never emits it, maxUploadBytes stays undefined and useUpload's cap
guard (useUpload.ts:56) is skipped ? the browser always proceeds to chunk-and-upload, so nothing
pre-rejects the 2.68 GB file client-side."
}
],
"deployed_vs_master": "Deployed OLD build (engine 0.1.0): uploads.py logic is byte-identical
to master for the size checks ? the whole-file 4096 MB / per-part 95 MB caps and their 413s have
existed unchanged since #99 (verified via git show 5eaba74 and e67add1), so the deployed build
413s at exactly the same place master would. Two differences, neither of which changes the
diagnosis: (1) the deployed build PREDATES the free-space guard (#407, f28af73) ? its
/upload/init has NO require_free_bytes call, so the deployed build has no latent 507 at all; the
ONLY failure it can produce here is the edge 413. (2) Its capabilities JSON omits
max_upload_mb/max_part_mb ? but current master ALSO omits them from the /api/capabilities return
(backend/main.py:379), so the SPA's instant client-side reject is inert against BOTH builds and
the browser always attempts the chunked upload. Net: the 413 the user sees is the Cloud Run
HTTP/1 ~32 MiB request-body limit rejecting a 95 MB part; it is identical on the deployed old
build and on master. The free-space 507 is a master-only concern that a fix must also address
once parts get through.",
"confidence": "high",
"fix_options": [
  "Immediate (no redeploy of the image): the 413 is an EDGE limit, not an app limit. Shrink
the part size below the Cloud Run HTTP/1 ~32 MiB request cap AND below Cloudflare's 100 MB ? set
security.max_part_mb to ~24-30 (e.g. 24) in the deployed config.json (gen_service_yaml
build_config security block). /upload/init then advertises 24 MB parts, the SPA chunks
accordingly, and each PUT slips under the ingress cap. This alone unblocks the upload path to
the app; verify against whichever proxy is in front (Cloud Run GFE and, if present,
Cloudflare).",
  "Enable HTTP/2 end-to-end on the control-plane service (add the run.googleapis.com/use-http2
annotation / --use-http2) so Cloud Run streams request bodies and the ~32 MiB per-request cap no
longer applies to a 95 MB part. Combine with the small-part option for defense in depth; confirm
Starlette/uvicorn handle h2c as expected.",
  "Fix the RAM-backed /tmp ceiling that bites AFTER parts get through: the 4Gi control-plane
service cannot hold a 2.68 GB assembled file in tmpfs (and master's require_free_bytes would 507
on it). Either raise the service --memory to comfortably exceed the largest expected upload
(README suggests ~6 GiB tmpfs for a 5.8 GB file ? so 8Gi+), or point upload_dir at a non-tmpfs
volume (a mounted GCS FUSE volume / persistent disk) so the staged file isn't memory-bound.",
  "Best long-term: stop staging the whole file on the control plane at all. Have the browser
upload DIRECT TO GCS via a resumable/signed-URL upload (or stream parts straight through to a
GCS resumable session), then hand the gs:// URI to /api/process (input_backend is already gcs,
input_root gs://khelsutra-rally-serving/videos). This removes both the edge request-size limit
and the 4Gi tmpfs ceiling, and matches the existing bucket-based compute path ? the 32Gi L4 JOB
pulls from GCS anyway.",
  "Make the client-side pre-flight actually work + surface the real limit: add max_upload_mb
AND max_part_mb (and ideally an effective edge/tmpfs-derived cap) to the /api/capabilities
response in backend/main.py so IngestPanel's instant reject and the SPA's chunk sizing reflect

```

reality instead of silently defaulting; today the field is declared in types.ts but never emitted, so the guard is dead code.",

"Raising security.max_upload_mb is NOT a fix here (2.68 GB is already under the 4096 MB cap); it would only matter for files >4 GB and would make the tmpfs problem worse. The binding constraints are the edge per-request size and the RAM-backed staging dir, not the whole-file cap."

```
],
"open_questions": [
  "Confirm the exact proxy topology in front of the deployed control plane: is Cloudflare Access actually in the request path for the upload origin, or is the browser hitting the Cloud Run *.run.app URL directly? This decides whether the binding edge cap is Cloudflare's 100 MB (parts already fit) or Cloud Run's ~32 MiB HTTP/1 request limit (parts do NOT fit) ? the evidence points to the latter, but a captured response header (server: Google Frontend vs cloudflare) on the failing PUT would nail it.",
  "Capture the actual 413 response headers/body from a failing part PUT on the hosted alpha to distinguish an edge 413 (HTML/plain, no FastAPI {detail}) from an app 413 (JSON {detail: 'Part exceeds ... per-part limit'}). The trace predicts an EDGE 413 with no FastAPI detail body.",
  "Verify the deployed old build's /upload/init did not have require_free_bytes (it predates #407) ? if the live build is actually newer than believed and DOES have the guard, a fixed edge/part-size issue would then surface as a 507 on the 4Gi tmpfs, which the memory/direct-to-GCS fixes must also cover."
]
}
```

Independently re-check it against the ENGINE code (backend/api/uploads.py, backend/config/models.py, free-space checks). Try to REFUTE it. Is the cited 413 path real and would it actually fire for a 2.68 GB file under a 4096 MB cap? Is the free-space-on-RAM-/tmp reasoning sound, or is /tmp not actually where parts land / not RAM-backed in the code? Return CONFIRMED / PARTLY_CONFIRMED / REFUTED with your own file:line reasoning.

2. assistant (2026-07-05T06:58:08.439Z)

I'll independently verify this root cause analysis against the actual engine code. Let me start by pulling the repo and reading the cited files.

3. user (2026-07-05T06:58:11.226Z)

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
64fa4a2 docs(a13): alpha launch review ? conditional-GO recommendation; GO/NO-GO to owner (#478)

4. user (2026-07-05T06:58:11.233Z)

```
1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5      under security.upload_dir, then submitted to /api/process by the returned path.
6      Upload state is in-process by design (this is the single-box alpha): a server restart
7      aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8      arrives with PR-3 identity scoping.
9
10     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11     ``upload_cfg`` resolves it through the ``backend.main`` module object so it always
12     reads the live value the server set at startup (and so tests that monkeypatch
13     ``backend.main.global_config`` flow through unchanged).
14     """
15
16     import os
17     import threading
18     import unicodedata
19     import uuid
20     from typing import Any, Dict, Optional
```

```

21
22 from fastapi import APIRouter, HTTPException, Request
23 from pydantic import BaseModel
24
25 from backend.utils.app_home import (
26     resolve_output_dir, # P0a: app-home-aware path resolution
27 )
28 from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
29 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30 from backend.utils.paths import safe_output_filename
31
32 router = APIRouter()
33
34 _uploads: Dict[str, Dict[str, Any]] = {}
35 _uploads_lock = threading.Lock()
36
37
38 class UploadInitRequest(BaseModel):
39     filename: str
40     total_size_bytes: Optional[int] = None
41
42
43 class UploadCompleteRequest(BaseModel):
44     total_parts: int
45
46
47 def _upload_cfg():
48     # Resolved lazily (import inside the function) to read the LIVE startup config that
49     # main() sets on the module global, and to avoid a circular import at module load
50     # (main.py imports this router; this module must not import main at import time).
51     import backend.main as _main
52
53     sec = getattr(_main.global_config, "security", None)
54     upload_dir = resolve_output_dir(
55         getattr(sec, "upload_dir", None) or "output/uploads"
56     ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
57     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
58     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
59     exts = [
60         e.lower().lstrip(".")
61         for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
62     ]
63     return upload_dir, max_bytes, part_bytes, exts
64
65
66 def _abort_upload(upload_id: str) -> None:
67     with _uploads_lock:
68         st = _uploads.pop(upload_id, None)
69         if st:
70             try:
71                 os.remove(st["tmp_path"])
72             except OSError:
73                 pass
74
75
76 def _normalize_upload_filename(filename: str) -> str:
77     """Normalize browser-provided file names before extension validation.
78
79     Some mobile/file-provider pickers can hand us visually normal names with
80     compatibility
81     characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
82     existing
83     traversal/null-byte checks in ``safe_output_filename`` as the authority after this
84     cleanup.
85     """

```

```

83         return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
84
85
86 @router.post("/api/upload/init")
87 def upload_init(req: UploadInitRequest):
88     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
89     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
90     try:
91         name = safe_output_filename(_normalize_upload_filename(req.filename))
92     except ValueError as e:
93         raise HTTPException(status_code=400, detail=str(e)) from e
94     ext = os.path.splitext(name)[1].lower().lstrip(".")
95     if ext not in exts:
96         raise HTTPException(
97             status_code=400,
98             detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
99         )
100     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
101         raise HTTPException(
102             status_code=413,
103             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
104         )
105     os.makedirs(upload_dir, exist_ok=True)
106     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
107     # effort ? an
108     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
109     # (never blocks a
110     # legit upload); the per-part total cap in upload_part is the hard backstop either
111     # way.
112     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
113     upload_id = uuid.uuid4().hex
114     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
115     open(tmp_path, "wb").close()
116     with _uploads_lock:
117         _uploads[upload_id] = {
118             "filename": name,
119             "tmp_path": tmp_path,
120             "parts": 0,
121             "bytes": 0,
122         }
123     return {
124         "upload_id": upload_id,
125         "max_part_mb": part_bytes // (1024 * 1024),
126         "next_part": f"/api/upload/{upload_id}/part/0",
127     }
128
129 @router.put("/api/upload/{upload_id}/part/{index}")
130 async def upload_part(upload_id: str, index: int, request: Request):
131     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
132     _, max_bytes, part_bytes, _ = _upload_cfg()
133     with _uploads_lock:
134         st = _uploads.get(upload_id)
135         if st is None:
136             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
137         if index != st["parts"]:
138             raise HTTPException(
139                 status_code=409,
140                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are sequential)",
141             )
142     received = 0
143     overflow = None
144     with open(st["tmp_path"], "ab") as fh:

```

```

144         async for chunk in request.stream():
145             received += len(chunk)
146             if received > part_bytes:
147                 overflow = (
148                     f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
149                 )
150                 break
151             if st["bytes"] + received > max_bytes:
152                 overflow = (
153                     f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
154                 )
155                 break
156             fh.write(chunk)
157     if overflow:
158         _abort_upload(upload_id)
159         raise HTTPException(
160             status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
161         )
162
163     with _uploads_lock:
164         st["parts"] += 1
165         st["bytes"] += received
166     return {
167         "upload_id": upload_id,
168         "received_parts": st["parts"],
169         "received_bytes": st["bytes"],
170     }
171
172
173 @router.post("/api/upload/{upload_id}/complete")
174 def upload_complete(upload_id: str, req: UploadCompleteRequest):
175     """Finalize: verify part count, move into place, return {path, video_id} for
176 /api/process."""
177     upload_dir, _, _, _ = _upload_cfg()
178     with _uploads_lock:
179         st = _uploads.get(upload_id)
180     if st is None:
181         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
182     if st["parts"] == 0 or st["parts"] != req.total_parts:
183         raise HTTPException(
184             status_code=409,
185             detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
186         )
187     final_path = os.path.join(upload_dir, st["filename"])
188     if os.path.exists(
189         final_path
190     ): # never overwrite an earlier upload with the same name
191         stem, ext = os.path.splitext(st["filename"])
192         final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
193     os.replace(st["tmp_path"], final_path)
194     with _uploads_lock:
195         _uploads.pop(upload_id, None)
196     video_id = compute_video_id(final_path)
197     return {
198         "path": os.path.abspath(final_path),
199         "video_id": video_id,
200         "size_bytes": os.path.getsize(final_path),
201         "next": "POST /api/process with this path",
202     }
203
204 @router.delete("/api/upload/{upload_id}")
205 def upload_abort(upload_id: str):
206     """Abort an in-flight upload and delete its partial file."""
207     with _uploads_lock:

```

```

208         known = upload_id in _uploads
209     if not known:
210         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
211     _abort_upload(upload_id)
212     return {"status": "aborted", "upload_id": upload_id}
213

```

5. assistant (2026-07-05T06:58:13.720Z)

Now let me verify the config models for the default values and check the free-space guard implementation.

6. user (2026-07-05T06:58:15.238Z)

```

1     """Free-space guards for the local box (STUDIO_MEDIA_LIFECYCLE.md P2, D11).
2
3     The single-box appliance fills its disk in three peaks: a chunked **upload** (raw file
into
4     ``upload_dir``), a **fetch-to-scratch** of a bucket/Drive source (a full local copy
under the temp
5     dir), and ? the real balloon ? **WSL frame extraction**, which can materialise ~15× the
source in
6     PNGs. D11: BLOCK on a local/WSL shortfall (buckets are warn-only elsewhere ? they aren't
7     capacity-bounded from here).
8
9     ``require_free_bytes`` is the HTTP-boundary guard: it raises **HTTP 507 (Insufficient
Storage)**
10    when the filesystem holding ``path`` can't hold ``need`` bytes, so the client fails fast
instead of
11    crashing a job mid-flight. It is best-effort about *measuring* (an unreadable
``disk_usage`` never
12    blocks ? inability to measure must not deny a legitimate request), but strict once a
real shortfall
13    is seen. The WSL frame-stage guard lives on ``wslCommandMixin`` (a different filesystem,
measured via
14    ``df`` inside WSL, not ``shutil.disk_usage``) ? see ``pipeline/detectors/base.py``.
15    """
16
17    from __future__ import annotations
18
19    import logging
20    import shutil
21    from typing import Optional
22
23    from fastapi import HTTPException
24
25    logger = logging.getLogger(__name__)
26
27
28    def free_bytes(path: str) -> Optional[int]:
29        """Best-effort free bytes on the filesystem holding ``path`` (``None`` if
undeterminable).
30
31        Never raises. ``shutil.disk_usage`` wants an existing path; we walk up to the
nearest existing
32        ancestor (an ``upload_dir``/scratch target may not exist yet at guard time)."""
33        import os.path
34
35        probe = path
36        for _ in range(64): # bounded ancestor walk (defensive against a pathological path)
37            if not probe:
38                return None
39            try:
40                return int(shutil.disk_usage(probe).free)
41            except (OSError, ValueError):

```

```

42         parent = os.path.dirname(probe.rstrip("/\\"))
43         if parent == probe: # reached the root and still can't stat ? give up
44             return None
45         probe = parent
46     return None
47
48
49     def require_free_bytes(
50         path: str, need: Optional[int], *, stage: str = "write"
51     ) -> None:
52         """Raise HTTP 507 if ``path``'s filesystem can't hold ``need`` bytes; otherwise
53         return.
54         ``need`` of ``None`` or ``<= 0`` is a no-op (nothing to reserve). Measuring is best-
55         effort: an undeterminable free-space read logs a warning and proceeds (we never deny on
56         inability to measure). A confirmed shortfall raises so the caller fails fast with an actionable
57         507."""
58         if not need or need <= 0:
59             return
60         free = free_bytes(path)
61         if free is None:
62             logger.warning(
63                 "free-space guard (%s): could not read free space at %s ? proceeding.",
64                 stage,
65                 path,
66             )
67             return
68         if free < need:
69             need_mb = need / (1024 * 1024)
70             free_mb = free / (1024 * 1024)
71             logger.warning(
72                 "free-space guard (%s): %s needs %.0f MB but only %.0f MB free ? refusing
73                 (507).",
74                 stage,
75                 path,
76                 need_mb,
77                 free_mb,
78             )
79             raise HTTPException(
80                 status_code=507,
81                 detail=(
82                     f"Not enough free disk space for {stage}: need ~{need_mb:.0f} MB, "
83                     f"only ~{free_mb:.0f} MB free. Free some space and retry."
84                 ),
85             )

```

7. user (2026-07-05T06:58:16.758Z)

```

backend/config/models.py:370: max_upload_mb: int = 4096
backend/config/models.py:371: max_part_mb: int = 95
Binary file backend/config/__pycache__/models.cpython-313.pyc matches

```

8. assistant (2026-07-05T06:58:22.651Z)

Config defaults confirmed: `max\_upload\_mb=4096`, `max\_part\_mb=95`. Now let me verify the deployment manifest claims (HTTP/2 annotation, memory, upload dir) and the capabilities response.

9. user (2026-07-05T06:58:24.159Z)

```

1         """Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING

```

```

M8/M9, #473).
2
3     The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4     session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5     clone. It emits a Knative service manifest for::
6
7         python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8         gcloud run services replace service.yaml --region asia-southeast1
9
10    Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11    a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12    cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13    gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15    The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16    it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17    server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18    backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19    identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20    service faces testers again.
21
22    Deliberate manifest choices (learned on the A1 deploy):
23    * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24    on a background thread; without always-allocated CPU a 202'd compile stalls (README
$3a).
25    * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26    instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27    once state is shared.
28    * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
29    with it; the code default ("rally-worker") is already correct.
30    * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31    committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32    `skip_ai_handoff=true` and never needs it).
33    ""
34
35    from __future__ import annotations
36
37    import argparse
38    import base64
39    import json
40    import sys
41
42    DEFAULT_PROJECT = "gen-lang-client-0306026826"
43    DEFAULT_REGION = "asia-southeast1"
44    DEFAULT_SERVICE = "rally-control-plane"
45    DEFAULT_SERVING_BUCKET = "gs://khehsutra-rally-serving"
46    DEFAULT_CF_TEAM_DOMAIN = "https://khehsutra.cloudflareaccess.com"
47    DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"
48
49

```

```

50 def build_config(args: argparse.Namespace) -> dict:
51     """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
52     bucket = args.serving_bucket.rstrip("/")
53     return {
54         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
55         # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
56         "indexing": {"skip_ai_handoff": True},
57         "storage": {
58             "backend": "gcs",
59             "output_root": bucket,
60             "input_backend": "gcs",
61             "input_root": f"{bucket}/videos",
62         },
63         "security": {
64             "edge_auth_enabled": args.edge_auth == "on",
65             "cf_team_domain": args.cf_team_domain,
66             "cf_access_aud": args.cf_access_aud,
67             "allowed_video_root": "/tmp/apphome/output",
68         },
69     }
70
71
72 def build_yaml(args: argparse.Namespace) -> str:
73     """Render the Knative service manifest for `gcloud run services replace`."""
74     cfg_b64 = base64.b64encode(
75         json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
76     ).decode("ascii")
77     # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
the
78     # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
79     wrapper = (
80         "mkdir -p /tmp/apphome && "
81         f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
82         "export RALLY_APP_HOME=/tmp/apphome && "
83         "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
84     )
85     return f"""\
86 apiVersion: serving.knative.dev/v1
87 kind: Service
88 metadata:
89   name: {args.service}
90   labels:
91     cloud.googleapis.com/location: {args.region}
92 spec:
93   template:
94     metadata:
95       annotations:
96         autoscaling.knative.dev/minScale: "{args.min_instances}"
97         autoscaling.knative.dev/maxScale: "{args.max_instances}"
98         run.googleapis.com/cpu-throttling: "false"
99   spec:
100     timeoutSeconds: {args.timeout}
101     containers:
102     - image: {args.image}
103       command: ["/bin/bash"]
104       args: ["-c", "{wrapper}"]
105       ports:
106       - containerPort: 8080
107     resources:
108       limits:
109         cpu: "{args.cpu}"
110         memory: {args.memory}
111     env:
112     - name: DEPLOYMENT_PROFILE

```

```

113         value: cloud-serving
114     - name: CLOUD_RUN_REGION
115       value: {args.region}
116     - name: GOOGLE_CLOUD_PROJECT
117       value: {args.project}
118     - name: RALLY_ALLOWED_HOSTS
119       value: "*"
120 """
121
122
123 def main(argv: "list[str] | None" = None) -> int:
124     p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
125     p.add_argument(
126         "--edge-auth",
127         choices=("on", "off"),
128         required=True,
129         help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
130 "
131         "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
132     )
133     p.add_argument("--project", default=DEFAULT_PROJECT)
134     p.add_argument("--region", default=DEFAULT_REGION)
135     p.add_argument("--service", default=DEFAULT_SERVICE)
136     p.add_argument(
137         "--image",
138         default=None,
139         help="Container image; default = the project's rally-worker:latest (post-#470 it
140 "
141         "carries cloud-run + auth, so the worker image IS the control-plane image).",
142     )
143     p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
144     p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
145     p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
146     p.add_argument("--min-instances", type=int, default=1)
147     p.add_argument(
148         "--max-instances",
149         type=int,
150         default=1,
151         help="Default 1: per-instance SQLite state splits across instances (issue
152 #471).",
153     )
154     p.add_argument("--cpu", type=int, default=2)
155     p.add_argument("--memory", default="4Gi")
156     p.add_argument("--timeout", type=int, default=300)
157     p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
158 stdout.")
159     args = p.parse_args(argv)
160     if args.image is None:
161         args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
162 worker:latest"
163     text = build_yaml(args)
164     if args.output == "-":
165         sys.stdout.write(text)
166     else:
167         with open(args.output, "w", encoding="utf-8", newline="\n") as f:
168             f.write(text)
169     return 0
170
171 if __name__ == "__main__":
172     raise SystemExit(main())

```

```

16:it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before exec'ing
the
51:     ""The app config the wrapper materializes at RALLY_APP_HOME/config.json.""
67:     "allowed_video_root": "/tmp/apphome/output",
80:     "mkdir -p /tmp/apphome && "
81:     f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
82:     "export RALLY_APP_HOME=/tmp/apphome && "
110:         memory: {args.memory}
152:     p.add_argument("--memory", default="4Gi")

```

11. assistant (2026-07-05T06:58:30.967Z)

Key findings on the manifest. Note the finder cited specific line numbers (85, 152, 82, 47) that don't all match current master ? the manifest at line 85 is `return f"""\` (start of YAML), and there is NO `run.googleapis.com/use-http2` annotation anywhere (confirmed: grep returned nothing). Memory default `4Gi` is at line 152. RALLY_APP_HOME=/tmp/apphome confirmed at line 82. Let me verify the upload_dir resolution (does it actually land under /tmp/apphome?) and the capabilities response.

12. user (2026-07-05T06:58:32.972Z)

```

1     ""Resolved application-data HOME for a packaged, batteries-included install.
2
3     PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4     the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5     ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6     the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7     "loses" their index. This module introduces ONE resolver every state path roots on.
8
9     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10    * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
sets it).
11    * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13    The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/local/share``)
14    is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15    calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16    this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17    real user profile during a test run).
18
19    stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20    the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21    ""
22
23    from __future__ import annotations
24
25    import os
26    import platform
27    from pathlib import Path
28
29    #: Environment variable the packaged launcher sets to pin the app-data home.
30    APP_HOME_ENV = "RALLY_APP_HOME"
31
32    #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33    APP_DIR_NAME = "Khelsutra"
34

```

```

35
36     def app_home() -> Path:
37         """The resolved application-data home.
38
39         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41         """
42         env = os.environ.get(APP_HOME_ENV)
43         if env and env.strip():
44             return Path(env.strip()).expanduser()
45         return Path(".")
46
47
48     def app_path(*parts: str | os.PathLike[str]) -> Path:
49         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50         return app_home().joinpath(*[str(p) for p in parts])
51
52
53     def default_config_path() -> Path:
54         """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """``<app-home>/env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63         return app_home() / ".env"
64
65
66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if (
76             str(home) == "."
77         ): # env unset (or pinned to CWD) ? historical load_dotenv() search
78             return None
79         candidate = home / ".env"
80         return candidate if candidate.exists() else None
81
82
83     def resolve_output_dir(output_dir: str) -> str:
84         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
85
86         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
87         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib

```

```

88     normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
89     ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
90     app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes
91     through untouched.
92     ""
93     home = app_home()
94     if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
95         return output_dir or "output"
96     if not output_dir:
97         return str(home / "output")
98     if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
99         return output_dir
100     return str(home / output_dir)
101
102
103 def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
104     """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
105     ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
106
107     * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
108     * macOS   ? ``~/Library/Application Support/<app_name>``
109     * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
110     ""
111     system = platform.system()
112     if system == "Windows":
113         base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
114     elif system == "Darwin":
115         base = str(Path.home() / "Library" / "Application Support")
116     else:
117         base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
118     return Path(base) / app_name
119

```

13. user (2026-07-05T06:58:34.929Z)

```

188:     Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
341: def _engine_version() -> str:
350: def api_version():
353:     "engine_version": _engine_version(),
354:     "api_version": API_VERSION,
360: def capabilities(request: Request):
380:     "api_version": API_VERSION,
381:     "engine_version": _engine_version(),
392:     "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),

```

14. assistant (2026-07-05T06:58:39.056Z)

This confirms a key point: with `RALLY\_APP\_HOME=/tmp/apphome` set (per the manifest) and the default `upload\_dir="output/uploads"` (relative), `resolve\_output\_dir` roots it under app_home ? `~/tmp/apphome/output/uploads`. That's on Cloud Run's RAM-backed `~/tmp`. The finder's claim on staging location holds. Let me read the full capabilities response to confirm the max_upload_mb/max_part_mb are NOT emitted.

15. user (2026-07-05T06:58:41.065Z)

```

341     def _engine_version() -> str:
342         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
343         try:

```

```

344         return _pkg_version("badminton-highlight-indexer")
345     except PackageNotFoundError:
346         return "unknown"
347
348
349     @app.get("/api/version")
350     def api_version():
351         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
352         return {
353             "engine_version": _engine_version(),
354             "api_version": API_VERSION,
355             "ui": _bundled_ui_version(),
356         }
357
358
359     @app.get("/api/capabilities")
360     def capabilities(request: Request):
361         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
362         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
363         value. Best-effort + never raises (the wizard degrades gracefully)."""
364         cfg = request.app.state.config
365         indexing = getattr(cfg, "indexing", None)
366         deployment = getattr(cfg, "deployment", None)
367         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
368         ffprobe = shutil.which(ffprobe_bin())
369         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
370         wasb = getattr(indexing, "wasb", None)
371         weights_path = (
372             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
373         )
374         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
375         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
376         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
377         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
378         nvidia_smi = shutil.which("nvidia-smi")
379         return {
380             "api_version": API_VERSION,
381             "engine_version": _engine_version(),
382             "segmenters": sorted(segmenter_registry.list_available().keys()),
383             "default_segmenter": getattr(indexing, "default_segmenter", None),
384             "ffmpeg": {
385                 "ffmpeg": bool(ffmpeg),
386                 "ffprobe": bool(ffprobe),
387                 "ffmpeg_path": ffmpeg,
388                 "ffprobe_path": ffprobe,
389             },
390             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
391             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
392             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
393             "deployment": {
394                 "profile": getattr(deployment, "profile", "") or "",
395                 "compute_target": getattr(deployment, "compute_target", "") or "",
396                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
397                 # the "run in the cloud" toggle only when true (else it's local-only).
398                 "cloud_available": _cloud_available(cfg),
399             },
400             # P3: which storage media THIS box can store INTO (local always; s3/gcs if

```

```

SDK+config;
401         # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
402         "storage": storage_capabilities(
403             getattr(cfg, "storage", None), local_free_path=output_base(cfg)
404         ),
405     }
406
407
408     def _friendly_video_name(filepath: str) -> str:
409         """A human name for a video from its stored path/URI ? the filename stem
(``~/match_2026.mp4`` ?
410         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
```

16. user (2026-07-05T06:58:41.591Z)

```

1     # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3     The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4     dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5     a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7     > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8     > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9     > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REPO=rally
20     # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21     REGION=asia-southeast1
22     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23     # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24     gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25     ```
26     *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27
28     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29     The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region
bucket**
30     mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
needed).
31     ```bash
32     gcloud run jobs create rally-worker \
33         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
34         --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
35         --cpu 8 --memory 32Gi \
36         --max-retries 0 --task-timeout 3600 \
```

```

37     --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
38     --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
39     --add-volume-mount volume=serving,mount-path=/gcs
40     ```
41     - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
42     the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
43     in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
44     `gcloud` that "did nothing" ? suspect a hidden prompt.
45     - 8 vCPU / 32 GiB: decode + x264 are CPU-side (NVDEC is a deferred D16 knob), so CPU
headroom is
46     what keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives
in memory,
47     so size `--memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of
tmpfs).
48
49     ## 3. Wire the dispatcher (control plane)
50     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
51     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
52     ```bash
53     export DEPLOYMENT_PROFILE=cloud-serving
54     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
55     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
56     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
57     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
58     ```
59     **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
60     on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
61     emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
62     0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
63     boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
64
65     ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
66
67     The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
68     request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
69     the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
70     progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
71
72     **The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
73     cloud-serving config.json into a startup wrapper, sets every flag below, pins
`maxScale=1`
74     (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB`
(a
75     reserved runtime env name that gets a manifest rejected; the code default is already
right):
76     ```bash
77     python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
78     gcloud run services replace service.yaml --region $REGION # from PowerShell on Windows
79     ```

```

```

80     `--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with
`on`
81     before testers touch the service again. The flag-based equivalent (for reference ? a
manifest
82     round-trips the bash wrapper's metacharacters, flags don't on Windows):
83     ```bash
84     gcloud run deploy rally-control-plane --image <image> --region $REGION \
85         --no-cpu-throttling \           # CPU stays allocated between requests ? the drain thread
finishes
86         --min-instances 1 \           # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
87         --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
88         --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving, CLOUD_RUN_REGION=$REGION, GOOGLE_CLOUD_PROJECT=$PROJECT
89     ```
90     Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
91     finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict
92     D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
93     the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
94     stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on
the next drain.
95     Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to
the CF proxy
96     (M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so
flipping
97     `security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify
works out of
98     the box.
99
100    ## 4. M7 acceptance (the owner runlog)
101    Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
102    `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
103    rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
104    exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
105
106    ## Gotchas
107    - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
108    + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
109    - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
110    ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
111    - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
112    real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
113    video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
114    (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
115    surfaces it as a `cloud dispatch error` job failure (not a traceback).
116    - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
117    - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
118    fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`);

```

```

119     an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
120     dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
121     though the SDK isn't a CI/test dependency.
122     - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split
plus its
123     alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser,
and a
124     `^:~` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in
the args,
125     e.g. `--args='^:~-v@@infer@@-video-uri=gs://?'`. (The dispatcher path doesn't have
this
126     problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must
include the
127     dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
128     `indexing.detector_impl=native`.
129     - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
130     `--add-volume-mount ?mount-path=/gcs` became `C:/Program Files/Git/gcs`, and
131     `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then
exec-fails).
132     Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks
gcloud's own
133     wrapper). PowerShell has its own trap: quote any flag value containing commas.
134     - **First execution after an image push** spends minutes in "Container image import in
progress"
135     (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
136     later executions reuse the imported image.
137

```

17. assistant (2026-07-05T06:58:47.589Z)

Confirmed: the `/api/capabilities` return dict (main.py lines 379-405) does NOT include `max\_upload\_mb` or `max\_part\_mb`. The finder's claim that the client-side pre-flight guard is inert on master holds. And the README (line 46-47) explicitly documents `/tmp` is RAM-backed, but note that comment is about the JOB (32Gi), not the control-plane SERVICE (4Gi) ? the finder's evidence correctly notes this distinction.

Now let me verify the two critical technical claims that determine the verdict:

1. Does the whole-file check at line 100 actually pass for 2.68 GB (i.e., is 2749 MB < 4096 MB)?
2. Is the ~32 MiB Cloud Run HTTP/1 request-body limit a real, correctly-characterized edge behavior?

Let me first confirm the arithmetic and re-check how the client sends parts, then examine whether the whole-file guard genuinely doesn't fire.

18. user (2026-07-05T06:58:53.885Z)

```

2.68 GB in MB (1024-based): 2744.3
max_upload_mb cap: 4096
whole-file check fires (2.68GB > 4096MB)? False | using 2749: False
parts at 95MB: 29
95 MB in MiB: 95 MB = 90.6 MiB
95 * 1024*1024 bytes = 99614720 bytes = 95.0 MiB
Cloud Run HTTP/1 request cap ~32 MiB in bytes: 33554432
part_bytes > 32MiB cap? True

```

19. user (2026-07-05T06:58:56.081Z)

```

khelsutra/web/src/api/client.ts
khelsutra/web/src/components/IngestPanel.tsx
---

```

```

77:    log("error", "api.http_error", { cid, method, url, status: res.status, body:

```

```

body.slice(0, 500) });
244:// A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge proxies
cap
251:   body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
281:   log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status, body:
body.slice(0, 500) });
289:export async function uploadComplete(uploadId: string, totalParts: number, cid?: string):
Promise<UploadCompleteResponse> {
292:   body: JSON.stringify({ total_parts: totalParts }),
318: // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math to
NaN (which
320: const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
321: const partBytes = partMb * 1024 * 1024;
322: const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
323: log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
326:   for (let i = 0; i < totalParts; i++) {
328:     const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
333:     const done = await uploadComplete(init.upload_id, totalParts, cid);

```

20. assistant (2026-07-05T06:59:00.522Z)

The arithmetic confirms the core mechanics. The whole-file check does NOT fire (2.68 GB ? 2744 MB < 4096 MB cap), so line 100's 413 is not the source. The SPA chunks at `init.max\_part\_mb` (95) ? 95 MiB parts (99,614,720 bytes each), which exceeds a 32 MiB HTTP/1 request cap. Let me verify the SPA sends `total\_size\_bytes` and read the exact client upload part / IngestPanel guard code.

21. user (2026-07-05T06:59:01.538Z)

```

244   // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
245   // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
246
247   /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[backend/api/uploads.py]. */
248   export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
249     return request<UploadInitResponse>("/upload/init", {
250       method: "POST",
251       body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
252       cid,
253     });
254   }
255
256   /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[backend/api/uploads.py]. */
257   export async function uploadPart(
258     uploadId: string,
259     index: number,
260     chunk: Blob,
261     cid?: string,
262     signal?: AbortSignal,
263   ): Promise<UploadPartResponse> {
264     const c = cid ?? newCorrelationId();
265     const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
266     log("info", "api.request", { cid: c, method: "PUT", url });
267     let res: Response;
268     try {
269       res = await fetch(url, {
270         method: "PUT",
271         body: chunk,
272         headers: { "content-type": "application/octet-stream", "x-correlation-id": c },

```

```

273         signal,
274     });
275     } catch (err) {
276         log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
277     });
278         throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
279     }
280     if (!res.ok) {
281         const body = await res.text().catch(() => "");
282         log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
283         body: body.slice(0, 500) });
284         throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
285         body });
286     }
287     log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
288     return (await res.json()) as UploadPartResponse;
289 }
290
291 /** Finalize: returns {path, video_id} ? feed `path` to processVideo()
292 [backend/api/uploads.py]. */
293 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
294 string): Promise<UploadCompleteResponse> {
295     return
296     request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
297         method: "POST",
298         body: JSON.stringify({ total_parts: totalParts }),
299         cid,
300     });
301 }
302
303 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
304 [backend/api/uploads.py]. */
305 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
306     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
307     cid });
308 }
309
310 export interface UploadProgress {
311     sentBytes: number;
312     totalBytes: number;
313 }
314
315 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
316 complete.
317 * On ANY failure or abort it best-effort DELETes the server-side partial, then
318 rethrows so the
319 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
320 video_id}.
321 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
322 recovery is a
323 * fresh init (re-pick the file), which the UI's retry does. */
324 export async function uploadFile(
325     file: File,
326     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
327     = {}),
328 ): Promise<UploadCompleteResponse> {
329     const cid = opts.cid ?? newCorrelationId();
330     const init = await uploadInit(file.name, file.size, cid);
331     // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
332     to NaN (which
333     // would silently upload zero parts) ? fall back to the engine's 95 MB default.
334     const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
335     init.max_part_mb : 95;
336     const partBytes = partMb * 1024 * 1024;
337     const totalParts = Math.max(1, Math.ceil(file.size / partBytes));

```

```

323     log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
324     let sent = 0;
325     try {
326         for (let i = 0; i < totalParts; i++) {
327             if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
328             const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
329             await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
330             sent += chunk.size;
331             opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
332         }
333         const done = await uploadComplete(init.upload_id, totalParts, cid);
334         log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
335         return done;
336     } catch (err) {
337         log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
338         await uploadAbort(init.upload_id, cid).catch(() => {});
339         throw err;
340     }
341 }
342

```

22. user (2026-07-05T06:59:03.979Z)

```

khelsutra/web/src/components/IngestPanel.tsx:73: const [maxUploadBytes, setMaxUploadBytes] =
useState<number | undefined>(undefined);
khelsutra/web/src/components/IngestPanel.tsx:82:         typeof c.max_upload_mb === "number" &&
c.max_upload_mb > 0 ? c.max_upload_mb * 1024 * 1024 : undefined,
khelsutra/web/src/components/IngestPanel.tsx:131:     }, maxUploadBytes);
khelsutra/web/src/hooks/useUpload.ts:20:export function useUpload(onComplete: (path: string,
filename: string) => void, maxUploadBytes?: number) {
khelsutra/web/src/hooks/useUpload.ts:28: const maxUploadBytesRef = useRef(maxUploadBytes);
khelsutra/web/src/hooks/useUpload.ts:29: maxUploadBytesRef.current = maxUploadBytes;
khelsutra/web/src/hooks/useUpload.ts:55:     const cap = maxUploadBytesRef.current;
khelsutra/web/src/api/types.ts:102: /** The largest upload THIS box accepts, in MB (engine
`security.max_upload_mb`, default 4096).
khelsutra/web/src/api/types.ts:106: max_upload_mb?: number;
khelsutra/web/src/api/types.ts:158: * ? max_part_mb, then `complete` returns a server-host
`path` to feed straight into POST /api/process. */
khelsutra/web/src/api/types.ts:162: max_part_mb: number;

```

23. assistant (2026-07-05T06:59:12.026Z)

This fully confirms the SPA side of the finder's analysis:

- Line 251: SPA sends `total\_size\_bytes` (so line-100 whole-file check IS exercised, and passes).
- Line 320-321: `partBytes = partMb \* 1024 \* 1024` where partMb comes from `init.max\_part\_mb` (95) ? 95 MiB parts.
- Line 328: `file.slice()` produces a fixed-size Blob ? `fetch(body: chunk)` sets a fixed Content-Length per part.
- IngestPanel line 82: reads `c.max\_upload\_mb` from capabilities; since the engine never emits it, `maxUploadBytes` stays `undefined` ? the client-side cap guard (useUpload.ts:55) is skipped.

Let me confirm the useUpload cap guard is indeed skipped when `maxUploadBytes` is undefined, and check the IngestPanel capabilities read.

24. user (2026-07-05T06:59:13.531Z)

```

50         return;
51     }
52     // Oversized file: fail INSTANTLY (no wasted round-trip) ? /upload/init would 413
anyway. Carry

```

```

53          // the size + cap so IngestPanel can localize a "trim/chunk it first" message. No
cap ? skip the
54          // guard (the engine still fail-fasts server-side).
55          const cap = maxUploadBytesRef.current;
56          if (cap && file.size > cap) {
57              setPhase({ status: "error", error: { code: "fileTooLarge", sizeBytes: file.size,
capBytes: cap } });
58              return;
59          }
60
61          const ctrl = new AbortController();
62          ctrlRef.current = ctrl;
63          const cid = newCorrelationId();
64          setPhase({ status: "uploading", pct: 0, filename: file.name });
65          log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
66
67          void (async () => {
68              try {
69                  const res = await uploadFile(file, {

```

25. user (2026-07-05T06:59:15.278Z)

```

366: # contain upload_dir or /api/process will reject the uploaded paths.
367: upload_dir: str = "output/uploads"
370: max_upload_mb: int = 4096
371: max_part_mb: int = 95

```

26. assistant (2026-07-05T06:59:22.721Z)

All confirmed. `useUpload.ts:56` ? the guard is `if (cap && file.size > cap)`, so with `cap` undefined it's skipped (browser always proceeds to chunk-and-upload). And `upload\_dir` default is `"output/uploads"` (relative), which under `RALLY\_APP\_HOME=/tmp/apphome` resolves to `/tmp/apphome/output/uploads` ? RAM-backed tmpfs on Cloud Run.

Now, the ONE claim I cannot verify from the codebase alone is the external infrastructure behavior: the "Cloud Run / GFE ~32 MiB HTTP/1 request-body limit." This is a Google Cloud platform fact, not a code fact. Let me assess it critically.

The finder's logical chain has one important nuance worth scrutinizing: the deployed control plane sits behind Cloudflare (edge_auth, cf_team_domain). The finder itself flags this as an open question. Let me think through both proxy scenarios and check whether there are any other engine-side 413 sources I might have missed.

27. user (2026-07-05T06:59:25.803Z)

```

backend/api/uploads.py:102:          status_code=413,
backend/api/uploads.py:160:          status_code=413, detail=overflow + " Upload aborted ? re-
init to retry."
backend/main.py:1033:# backend.main.global_config, so behavior ? route paths, status codes,
body-size limits,

```

28. user (2026-07-05T06:59:28.428Z)

```

35:from fastapi.middleware.cors import CORSMiddleware
44:from starlette.middleware.trustedhost import TrustedHostMiddleware
72:# because the module-level add_middleware(...) below calls
_cors_origins_from_env/_allowed_hosts_from_env
126:app.add_middleware(
127:    CORSMiddleware,
138:app.add_middleware(
139:    TrustedHostMiddleware,
452:def list_videos(request: Request, limit: Optional[int] = None):
459:    _video_summary(v) for v in request.app.state.db.list_videos(limit=limit)
1033:# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,

```

=== check for any reverse proxy / nginx / gunicorn config ===

29. assistant (2026-07-05T06:59:37.088Z)

This confirms there are only TWO 413 sources in the entire engine, both in `uploads.py` (lines 102 and 160), and neither fires for this scenario:

- Line 102 (whole-file): 2.68 GB < 4096 MB ? does not fire.
- Line 160 (per-part / running-total): a 95 MiB part exactly equals `part\_bytes` and never exceeds it; running total tops at ~2744 MB < 4096 MB ? does not fire.

There is no Starlette/uvicorn body-size middleware limiting request bodies (only CORS and TrustedHost middleware). Uvicorn itself has no default request-body-size cap. So the app CANNOT produce the 413 for this file ? confirming the 413 must originate upstream (edge/proxy), which is precisely the finder's conclusion.

The finder's evidence line-number citations to `gen\_service\_yaml.py` (line 85, 152, 82) are slightly off vs current master (the file structure differs ? the manifest f-string starts at line 85, memory at 152, RALLY_APP_HOME at 82), but the substantive claims are all correct: no `use-http2` annotation exists (grep confirmed zero matches), memory default is 4Gi, and RALLY_APP_HOME=/tmp/apphome.

Now the one external-infrastructure claim. Let me evaluate it rigorously since the verdict hinges on it.

30. user (2026-07-05T06:59:45.642Z)

=== Proxy scenario analysis ===

Part size the SPA sends: 95 MiB = 99614720 bytes

Scenario A: Cloudflare in path (edge_auth_enabled, cf_team_domain set)
Cloudflare free/pro request-body limit: 100 MB
95 MiB = 99.6 MB (decimal) ? UNDER 100 MB ? part would PASS Cloudflare
=> If ONLY Cloudflare, 95 MiB parts do NOT 413

Scenario B: Cloud Run *.run.app direct (HTTP/1, no use-http2 annotation)
Cloud Run HTTP/1 request limit: 32 MiB = 33554432 bytes
95 MiB > 32 MiB ? part 413s at the Google Front End
=> Cloud Run HTTP/1 32MiB cap fires on part 0

NOTE: even WITH Cloudflare, the request still terminates at Cloud Run behind it, so the Cloud Run 32 MiB HTTP/1 cap applies REGARDLESS of whether CF fronts it (CF forwards the body to the origin, which is Cloud Run over HTTP/1).

31. assistant (2026-07-05T06:59:54.982Z)

My independent analysis strengthens (rather than weakens) the finder's conclusion on the edge 413: whether or not Cloudflare fronts the service, the request ultimately terminates at Cloud Run over HTTP/1 (no `use-http2` annotation), and a 95 MiB part exceeds Cloud Run's ~32 MiB HTTP/1 request-body limit. The Cloudflare 100 MB cap is a near-miss (95 MiB ? 99.6 MB decimal, just under), so Cloudflare alone wouldn't 413 ? but the Cloud Run cap behind it would. This means the finder's diagnosis is robust to the proxy-topology open question: the binding constraint is Cloud Run's HTTP/1 body limit regardless.

One caveat to note precisely: the exact numeric value of Cloud Run's HTTP/1 request limit ("~32 MiB") is the finder's characterization of a Google platform behavior. The precise number is a platform detail I cannot verify from code, but (a) it is well below the 95 MiB part size by a wide margin, and (b) the app itself provably cannot emit this 413, so the upstream-413 conclusion stands even if the exact cap differs somewhat.

Let me do a final check on the free-space/tmpfs reasoning to confirm the 507 characterization and that it's a latent secondary (not the observed) failure.

32. user (2026-07-05T06:59:58.014Z)

```
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
=== confirm require_free_bytes called only at init with total_size_bytes ===
28:from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
40:    total_size_bytes: Optional[int] = None
100:    if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
107:    # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard (never
blocks a
109:    require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
```

33. assistant (2026-07-05T07:00:06.641Z)

Confirmed. The free-space guard (`require\_free\_bytes`, freespace.py:77) raises 507, never 413, and is called at `/upload/init` with `req.total\_size\_bytes` (uploads.py:109). On the 4Gi control-plane with RAM-backed `/tmp`, `shutil.disk\_usage` would report memory-bound free space, and a 2.68 GB reservation could plausibly 507. But critically: (a) it's a 507, not the observed 413, so it's not the source; and (b) it fires at `/upload/init` ? BEFORE any part PUT ? so on current master, if the guard triggered, the user would see a 507 at init and never even reach the part uploads. This is a genuine latent second failure on master that a fix must also address, exactly as the finder states.

The guard was added in #407 (f28af73), consistent with the finder's claim that the deployed old build (engine 0.1.0) predates it. I can't verify the deployed build's exact commit from this checkout, but the git history supports the claim's plausibility. This is a "deployed-vs-master" nuance the finder itself flagged and does not affect the primary diagnosis of the observed 413.

I have completed my independent verification. Let me record my verdict.

34. user (2026-07-05T07:00:44.377Z)

Structured output provided successfully